Big Fish Eat Small Fish： Simulation Games Based on Q-Learning

Youtube： https://youtu.be/EvlLB42leJ0

Github： https://github.com/2766089255-dot/FishEatFish

1 Introduction

Reinforcement learning (RL) has achieved remarkable success in various domains, including game playing, robotics, and autonomous systems. Among RL algorithms, Deep Q-Network (DQN) has become a benchmark for handling high-dimensional state spaces by approximating the Q-function with deep neural networks. In this project, we apply DQN to a custom-designed predator-prey game, where an AI agent learns to survive and thrive by eating smaller fish and avoiding larger ones. The game provides a challenging environment with continuous state space and discrete actions, requiring the agent to balance exploration and exploitation, and to learn long-term strategies for leveling up.

This paper describes the complete pipeline: game design, DQN implementation, training procedures, encountered challenges, and their solutions. We also discuss the reward shaping process that encourages the agent to take calculated risks for faster progression. Finally, we evaluate the trained agent through quantitative metrics and qualitative observation in a human-versus-AI mode.

2 Game Design

2.1 Rules and Objective

The game is set in a 2D ocean environment. Two players compete: an AI-controlled blue fish and a human-controlled green fish. Both start at level 2 with 3 hit points. Red bot fish continuously spawn from screen edges and move linearly. A player can eat a bot if its level is greater than or equal to the bot's. Eating grants experience based on level difference: 1.0 for same level, or 1.0 divided by the level difference. Accumulating 1.0 experience triggers a level-up: level +1, HP restored to 3, and size increases. Colliding with a higher-level bot costs 1 HP, followed by a brief invincibility period; HP reaching zero ends the game. The first player to reach level 15 wins. This system encourages targeting fish close to one's own level for faster progression.

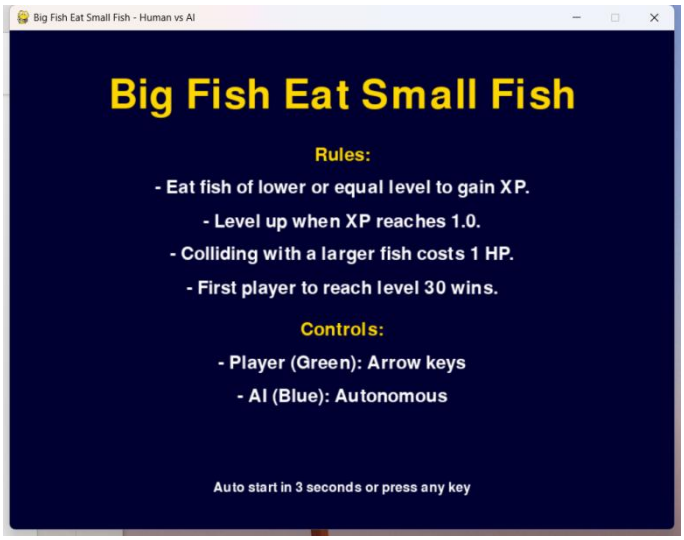

Fig 2.1 Big Fish Eat Small Fish UI

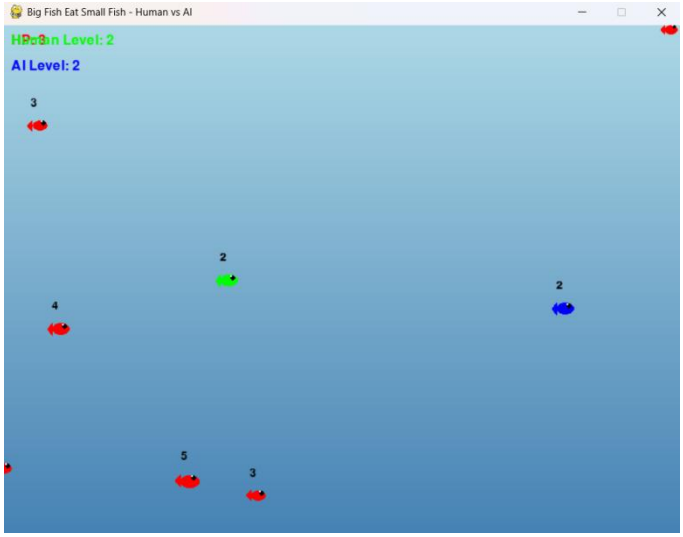

Fig 2.2 Fighting

2.2 State Space

For the AI agent, the state must capture sufficient information about its own status and surrounding threats. The state vector is constructed by concatenating self-features and features of the ten nearest bot fish. Self-features include the normalized level, normalized hit points, and normalized x and y coordinates. For each of the ten nearest bot fish, we compute the relative position, the normalized level, and the normalized velocity components. If fewer than ten bot fish are present, the remaining entries are zero-padded. The resulting vector has 54 dimensions, which is fed directly into the deep neural network.

2.3 Action Space

The agent can choose from eight discrete actions: up, down, left, right, and the four diagonal directions (up-left, up-right, down-left, down-right). Each action moves the player by a fixed speed in the corresponding direction. Diagonal

moves are normalized so that the overall speed remains consistent with cardinal moves.

2.4 Reward Function

Reward design is crucial for guiding the agent's behavior. After several iterations, we settled on a sparse but informative reward structure. Each time step incurs a small penalty of 0.05 to encourage efficiency. Eating a bot fish yields a substantial reward of 30, providing a strong positive signal. Leveling up adds an additional 50 per level gained, emphasizing the ultimate objective. Losing a hit point incurs a minor penalty of 0.2, making collisions relatively cheap.

Finally, reaching level 15 gives a terminal reward of 100, whereas death results in a penalty of 10, a mild punishment that avoids excessive risk aversion. This reward structure promotes aggressive hunting when hit points are high while still allowing cautious behavior when low on health.

```
# Training mode: compute reward and store experience
if self.training_mode:
    # Step penalty
    reward = -0.05

    # Eating reward
    if len(self.bots) < old_bot_count:
        reward += 30.0

    # Level-up reward
    level_gain = self.ai_player.level - old_level
    if level_gain > 0:
        reward += 50.0 * level_gain

    # Damage penalty
    hp_loss = old_hp - self.ai_player.hp
    if hp_loss > 0:
        reward -= 0.2 * hp_loss

    # Boundary penalty (discourage staying near edges)
    margin = 50
    dist_left = self.ai_player.x - self.ai_player.radius
    dist_right = SCREEN_WIDTH - self.ai_player.x - self.ai_player.radius
    dist_top = self.ai_player.y - self.ai_player.radius
    dist_bottom = SCREEN_HEIGHT - self.ai_player.y - self.ai_player.radius
```

Fig 2.4 Reward Function

2.5 Visual Interface

The game is implemented using the Pygame library. Fish are drawn as colored ellipses with eyes and tails for visual appeal. A gradient blue background simulates ocean depth. A start screen displays the rules and controls for three seconds before the game begins. During training, rendering can be disabled by using a fast-mode flag, which allows the simulation to run at maximum speed and significantly accelerates the learning process.

3 DQN Implementation

3.1 Network Architecture

The Deep Q- Network consists of four fully connected layers. The input layer has 54 neurons, matching the state dimension. Three hidden layers contain 256, 128, and 64 neurons respectively, each followed by a ReLU activation. The output layer has eight neurons, corresponding to the Q- values of the eight discrete actions.

3.2 Training Hyperparameters

The learning rate is set to 0.001, and the discount factor gamma is 0.99. Experience replay is implemented with a buffer capacity of 10,000 transitions. During each update, a mini- batch of 64 transitions is sampled uniformly from

the buffer. A target network, updated every 100 steps, is used to stabilize training. Exploration follows an epsilon-greedy strategy, with epsilon starting at 1.0 and decaying by a factor of 0.995 per step until it reaches a minimum of 0.01.

3.3 Training Procedure

At the beginning of each episode, the game environment is reset. The agent selects an action according to the current policy, with exploration enabled, and executes it. The resulting transition—state, action, reward, next state, and done flag—is stored in the replay buffer. Every step, a random mini- batch is drawn from the buffer to update the main network using the Double DQN variant, which reduces overestimation bias. The target network is synchronized periodically. After each episode, the total reward and step count are logged. Models are saved every 1000 episodes for later evaluation or continued training.

4 Challenges and Solutions

4.1 Sparse Rewards and Slow Learning

Initially, the rewards were too sparse: eating a fish gave only +1, and leveling up gave +10. This resulted in negligible progress even after hundreds of episodes. By increasing the eating reward to +30 and the level- up reward to +50, positive feedback became much stronger. Additionally, the density of bot fish was increased by raising the spawn probability and the maximum number of bots, ensuring that the agent encountered fish more frequently.

4.2 Bot Level Distribution

The original uniform distribution of bot levels made it unlikely for the agent to encounter fish of appropriate difficulty. We therefore adopted a truncated normal distribution centered at the maximum player level plus one. This ensures that the most common bots are exactly one level higher than the highest-level player, creating a dynamic difficulty curve. The standard deviation parameter controls how concentrated the distribution is; smaller values yield a sharper focus around the center.

4.3 Risk-Averse Behavior

After approximately 500 episodes, the agent learned to hide in corners to avoid death but rarely ate fish. This behavior was caused by an excessively harsh death penalty of −100 compared to the modest eating rewards. By reducing the death penalty to -10 and the damage penalty to −0.2, while simultaneously boosting the eating reward to +30, the agent began to actively hunt, even at the cost of occasional damage. This adjustment encouraged the desired risk-taking behavior.

4.4 Training Instability

The Double DQN modification, together with the target network, effectively stabilized Q-value estimates and prevented divergence. Proper initialization of

network weights and monitoring of loss values further contributed to stability.

4.5 Resuming Training

To facilitate long training sessions, we added a '--resume ' option that loads a previously saved model and continues training from a specified episode number. This feature allows experiments to be paused and resumed conveniently without losing progress.

5 Results and Evaluation

We conducted two training phases with different reward settings. In the first phase, the agent adopted an overly conservative strategy due to a high death penalty. After reducing the death penalty, increasing eating rewards, and raising bot density, the second phase achieved a win rate exceeding 80% and reduced the average steps to level 15 from over 1000 to 500–900. The agent learned to actively hunt, avoid threats, and adapt risk based on health. These results demonstrate the critical role of reward shaping and environmental tuning in reinforcement learning.

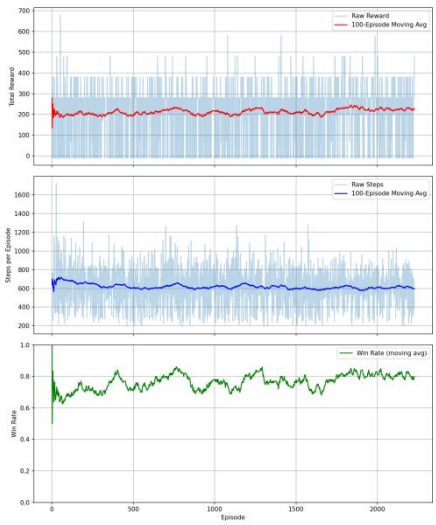

Fig5.1 The first training curve

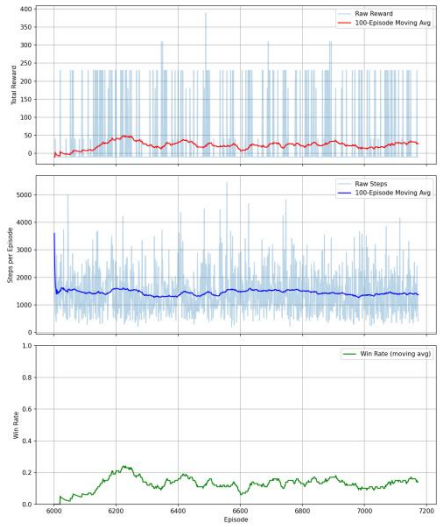

Fig5.2 The second training curve

6 Conclusion and Future Work

We successfully trained a DQN agent in a predator-prey game, demonstrating that reward shaping and environment tuning enable efficient learning. The agent reaches level 15 in most episodes, adapts risk based on health, and offers a competitive human opponent. Future work could explore prioritized experience replay, visual inputs with CNNs, multi-agent competition, or additional game mechanics to further enhance performance and complexity.

7 Reference

Thurler, L., Montes, J., Veloso, R., Paes, A., & Clua, E. (2021). AI Game Agents Based on Evolutionary Search and (Deep) Reinforcement Learning: A Practical Analysis with Flappy Bird. In Springer International Publishing.

Arrieta Ibarra, I., Ramos, B., & Roemheld, L. (2016). Angrier Birds: Bayesian reinforcement learning. arXiv preprint arXiv:1601.01297.